

Group 9: Database Module - Full Specification

Module Overview

Group: Database (Group 9)

Members: Yashvasin Sai Reddy Siddavarapu, Krist Veseli, Mitchell McAuley, Andrew Shilman, Ian Wettlaufer

Mission: Provide a comprehensive, high-performance data persistence layer that enables all groups to store, retrieve, and update complex game states, agent data, world configurations, and user interactions efficiently while minimizing storage footprint and network bandwidth.

Database Class - NOT YET IMPLEMENTED

The Database class is a **proposed high-level facade** that would provide a simple API for other groups to store and load objects without worrying about serialization, compression, or storage details.

Key idea: One-line store, one-line load.

Instead of this:

```
std::string serialized = serializer.Serialize(world);  
  
auto compressed = StringCompressor::CompressToBytes(serialized);  
  
dataGrid.Store("world", compressed);
```

Users would do this::

```
db.Store("world", world);
```

Public Member Functions (Templated):

bool Store(const std::string& key, const T& obj):

This function will serialize the object passed to it, call StringDiff and StringCompressor and store as Datum objects in a DataGrid. Overwrites the values if it already exists.

std::optional<T> Load(const std::string& key);

Loads an object from memory and decompresses, deserializes back to the original object.
Returns an empty `std::optional` if the object does not exist.

`bool Update(const std::string& key, const T& obj);`

Updates a value in memory based on a given key. Behaves identical to `Store()` if the object exists, otherwise it fails and returns false.

`bool Delete(const std::string& key);`

Deletes an object from memory based on a given key. Returns false if the key doesn't exist.

`bool Exists(const std::string& key);`

Checks for existence of an object based on a given key.

StringCompressor - Krist Veseli

StringCompressor implements the **LZW (Lempel-Ziv-Welch) compression algorithm** to reduce the storage footprint of serialized world states and agent logs before database storage. LZW is a lossless compression algorithm that works by building a dictionary of repeated patterns in the data, replacing them with shorter codes.

How LZW works:

1. Start with a dictionary of all 256 ASCII characters (codes 0-255)
 2. Read through input, finding the longest match in the dictionary
 3. Output the code for that match
 4. Add the match + next character to the dictionary as a new entry
 5. Dictionary grows up to 4096 entries, then stops expanding
 6. Decompression rebuilds the same dictionary from the codes
-

Error Enumeration

CompressorError

Defines error types for compression/decompression operations.

```
enum class CompressorError {
```

```
    EmptyInput,           // Empty vector passed to Decompress()
```

```

InvalidCode,      // Code sequence is invalid/corrupted

CorruptedHeader,  // Magic bytes or header structure invalid

VersionIncompatibility // Data from incompatible version

};

```

Error Descriptions:

Error	When It Occurs	How to Handle
EmptyInput	Tried to decompress an empty vector	Check data exists before decompressing
InvalidCode	Code references non-existent dictionary entry	Data is corrupted, load from backup
CorruptedHeader	Magic bytes wrong or incomplete header	File corrupted during storage/transfer
VersionIncompatibility	Version byte doesn't match current version	Need data migration or format upgrade

Public Member Functions

Compress

Signature:

```
static std::vector<uint16_t> Compress(const std::string& input);
```

Purpose: Convert a string into compressed codes using LZW algorithm.

Parameters:

- input - String to compress (can be any UTF-8 string)

Returns: Vector of uint16_t codes representing compressed data

- Empty vector if input is empty
- Codes range from 0-4095 (12-bit values in 16-bit storage)

Algorithm:

1. Initialize dictionary with 256 ASCII characters
 2. Read input character by character
 3. Find longest match in dictionary
 4. Output code for match
 5. Add match + next char to dictionary (up to 4096 entries)
 6. Repeat until input exhausted
-

Decompress

Signature:

```
static std::expected<std::string, CompressorError> Decompress(  
    const std::vector<uint16_t>& compressed);
```

Purpose: Reconstruct the original string from LZW codes.

Parameters:

- compressed - Vector of codes from Compress()

Returns:

- std::expected containing original string on success
- CompressorError on failure (empty input, invalid codes)

Algorithm:

1. Initialize dictionary with 256 ASCII characters
2. Read first code, output corresponding string
3. For each subsequent code:
 - If code exists in dictionary, output its string
 - If code doesn't exist, it must be previous + previous[0]
 - Add previous + current[0] to dictionary
4. Repeat until all codes processed

Error conditions:

- Returns EmptyInput if compressed vector is empty
 - Returns InvalidCode if code is out of valid range
-

CompressToBytes

Signature:

```
static std::vector<uint8_t> CompressToBytes(const std::string& input);
```

Purpose: Produce storage-ready compressed data with header for database persistence.

Parameters:

- `input` - String to compress

Returns: Vector of bytes in storage format:

Byte format details:

- Byte 0: Magic byte 1 (0x4C = 'L')
- Byte 1: Magic byte 2 (0x5A = 'Z')
- Byte 2: Version (0x01)
- Bytes 3+: Compressed codes (2 bytes per code, big-endian)

Why this format:

- Magic bytes identify data as LZW-compressed
 - Version allows future format changes
 - Prevents accidental decompression of non-compressed data
-

DecompressFromBytes

Signature:

```
static std::expected<std::string, CompressorError> DecompressFromBytes(  
    const std::vector<uint8_t>& data);
```

Purpose: Reconstruct original string from storage format with validation.

Parameters:

- data - Byte vector from storage (from CompressToBytes())

Returns:

- std::expected containing original string on success
- CompressorError on failure

Validation steps:

1. Check minimum size (5 bytes: header + at least 1 code)
2. Verify magic bytes (0x4C, 0x5A)
3. Verify version matches (0x01)
4. Check payload has even number of bytes (codes are 2 bytes each)
5. Convert bytes to codes (big-endian uint16_t)
6. Decompress codes to string

Error conditions:

- CorruptedHeader if size < 5, magic bytes wrong, or odd payload length
 - VersionIncompatibility if version byte doesn't match
 - InvalidCode if decompression fails (from Decompress())
-

GetCompressionRatio**Signature:**

```
static double GetCompressionRatio(const std::string& original,  
                                const std::vector<uint16_t>& compressed);
```

Purpose: Measure how much space was saved by compression.

Parameters:

- `original` - Original uncompressed string
- `compressed` - Compressed code vector from `Compress()`

Returns: Ratio as a double (0.0 to 1.0+)

- 0.6 = 60% of original size (40% space saved)
- 0.4 = 40% of original size (60% space saved)

- 1.0+ = No compression or expansion (shouldn't compress)

Calculation:

$$\text{ratio} = (\text{compressed_size_in_bytes}) / (\text{original_size_in_bytes})$$
$$= (\text{compressed.size()} * 2) / \text{original.size()}$$

IsCompressed

Signature:

```
static bool IsCompressed(const std::vector<uint8_t>& data);
```

Purpose: Determine if data was compressed by CompressToBytes().

Parameters:

- data - Byte vector to check

Returns:

- true if data appears to be compressed (has valid header)
- false otherwise

Detection criteria:

1. Size ≥ 5 bytes (minimum valid compressed data)
 2. First byte is 0x4C ('L')
 3. Second byte is 0x5A ('Z')
 4. Third byte is 0x01 (version 1)
-

Private Helper Functions

InitializeCompressionDict

Signature:

```
static void InitializeCompressionDict(std::unordered_map<std::string, uint16_t>& dict);
```

Purpose: Populate dictionary with 256 basic ASCII characters.

Parameters:

- dict - Reference to dictionary map (string → code)

Called by: Compress() at the start of compression

InitializeDecompressionDict

Signature:

```
static void InitializeDecompressionDict(std::unordered_map<uint16_t, std::string>& dict);
```

Purpose: Populate reverse dictionary with 256 basic ASCII characters.

Parameters:

- dict - Reference to dictionary map (code → string)

Called by: Decompress() at the start of decompression

StringDiff - Andrew Shilman

StringDiff computes and applies “patches” between two strings so that systems can save and send incremental changes instead of full string copies. This is especially useful for serialized state snapshots and logs. It is designed to be:

- Fast - It uses a simple prefix/suffix heuristic
- Safe - Validates patches against the intended base string using a lightweight signature
- Non-throwing - Returns failures via `std::expected<..., DiffError>`

StringDiff is a static utility class, default constructor is deleted. The patch type is a `StringDiff::Diff`

Similar Standard Library Concepts

- `Std::string / substr`

- Std::expected
- Std::from_chars
- Std::ostringstream

Diff Patch Format (StringDiff::Diff)

A patch stores:

- Base_hash (uint64_t): Hash signature of base string
- Base_length (uint64_t): Base string length
- Prefix_length (uint64_t): Unchanged prefix length
- Suffix_length (uint64_t): Unchanged suffix length
- Replacement (std::string): Replacement middle segment from updated string

Invalid:

- Prefix_length <= base_length
- Suffix_length <= base_length - prefix_length

Diff Error:

- **BaseLengthMismatch:** ApplyDiff base length differs from patch base_length
- **BaseHashMismatch:** ApplyDiff base hash doesn't match patch base_hash
- **InvalidPatchInvariant:** Prefix/suffix violate the invalid bounds above
- **MalformedEncoding:** Encoded patch missing required separators / structure
- **InvalidNumberField:** Numeric field fails strict parsing (empty, overflow, or partial parse like "123abc")
- **ReplacementLengthMismatch:** Encoded replacement length doesn't match actual remaining payload
- **PatchTooLarge:** Replacement / encoded diff / reconstructed output violates size limits

Size Limits:

The limits prevent runaway allocations to keep browser/Emscripten behavior predictable

- **MaxReplacementSize:** 1 MiB

- **MaxEncodedDiffSize:** 1 MiB
- **MaxPatchedStringSize:** 8 MiB

Violations return `DiffError::PatchTooLarge`

Public Member Functions

MakeDiff(base, updated) -> Diff

Static `Diff MakeDiff(const std::string& base, const std::string& updated)`

Purpose: Create a patch that transforms base into updated using common prefix/suffix alg

ApplyDiff(base, patch) -> expected<string, DiffError>

Static `std::expected<std::string, DiffError> ApplyDiff(const std::string& base, const Diff& patch)`

Purpose: Validate a patch against the base string, then reconstruct the updated

EncodeDiff(patch) -> expected<string, DiffError>

Static `std::expected<std::string, DiffError> EncodeDiff(const Diff& patch)`

Purpose: Convert a Diff into a compact representation for storage (e.g., inside Datum / DataGrid, or to be serialized)

DecodeDiff(encoded) -> expected<Diff, DiffError>

Static `std::expected<Diff, DiffError> DecodeDiff(const std::string& encoded)`

Purpose: Parse the encoded patch string back into a Diff with strict validation

Private Helper Functions

ComputeHash(str) -> uint64_t

Purpose: Produce a signature hash for base string signature verification stuff

TryParseU64(begin, end, out) -> bool

Purpose: Strictly parse a uint64_t from a character range

Coordination Points

- Serializer: encoded diffs are plain strings and can be safely framed using Serializer's length-prefixed string encoding
- Datum / DataGrid: store encoded patches as strings in Datum cells for incremental state/log updates

Serializer - Yashvasin Siddavarapu

The serializer class provides text based serialization and deserialization for C++ primitive types, containers and user defined custom types. It was designed to persist game state to and from a flat string representation.

- Round trip fidelity : serialize then deserialize always produces the original value
- Composability : containers and custom types are built by composing primitive serializers recursively
- Extensibility : any team can register custom types at runtime via RegisterType without modifying Serializer source

Wire Format

Every serialized value starts with a type tag followed by the relevant information separated by colons.

Type	Format	Input	Output
------	--------	-------	--------

int	i:<value>;	42	i:42;
double	d:<value>;	3.14	d:3.14...;
bool	b:<0 1>;	true	b:1;
char	c:<ch>;	'Z'	c:Z;
string	s:<len>:<content>;	"hello"	s:5:hello;
vector	v:<size>:<elements>	{10, 20}	v:2:i:10;i:20;
map	m:<size>:<kv pairs>	{"a", 1}	m:1:s:1:a;i:1;
Unordered map	u:<size>:<kv pairs>	{"a", 1}	u:1:s:1:a;i:1;
custom	custom:<id>:<len>:<d>;	Agent{...}	custom:Agent:...;

Design notes :

- Double uses a max_digits10 precision to ensure lossless roundtrips
- Strings are length prefixed so embedded colons and semicolons are handled safely
- Containers encode their element count first, then each element is serialized inline using the same format recursively

Class Structure

Serializer.hpp : Header with class declaration. All template methods (vector, map, unordered_map, custom type, DeserializeAt) are defined inline.

Serializer.cpp : Implementation of non-template methods: primitive Serialize/Deserialize functions and positional parsers for int, double, bool, char, string.

Three private template structs enable compile-time container detection for the DeserializeAt dispatcher

Trait	Matches	Used By
is_vector<T>	std::vector<T, A>	DeserializeAt<T>
is_map<T>	std::map<K,V,C,A>	DeserializeAt<T>
is_unordered_map<T>	std::unordered_map<K,V,H,E,A>	DeserializeAt<T>

Custom Type Registry

A private std::unordered_map<std::string, TypeEntry> stores registered types. Each TypeEntry holds two type erased functions :

Field	Erased Signature	Purpose
serialize_fn	const void* → std::string	Converts object to serialized string

deserialize_fn	const std::string& → std::any	Reconstructs object from string; empty std::any on failure
----------------	-------------------------------	--

Public API Reference

Primitive Serialization : All methods are const and return std::string

```
std::string Serialize(int value) const;          // i:<value>;
std::string Serialize(double value) const;      // d:<value>; (max_digits10 precision)
std::string Serialize(bool value) const;        // b:1; or b:0;
std::string Serialize(char value) const;        // c:<ch>;
std::string Serialize(const char* value) const; // delegates to string overload
std::string Serialize(const std::string& value) const; // s:<len>:<content>;
```

Container Serialization

```
template <typename T>
std::string Serialize(const std::vector<T>& vec) const;

template <typename K, typename V>
std::string Serialize(const std::map<K, V>& m) const;

template <typename K, typename V>
std::string Serialize(const std::unordered_map<K, V>& m) const;
```

Deserialization : Top-Level, each method validates the type tag, pursues the value, and returns std::nullopt on any failure. Input must be the complex serialized token.

```
std::optional<int>      DeserializeInt(const std::string& data) const;
std::optional<double>   DeserializeDouble(const std::string& data) const;
std::optional<bool>     DeserializeBool(const std::string& data) const;
std::optional<char>     DeserializeChar(const std::string& data) const;
std::optional<std::string> DeserializeString(const std::string& data) const;
```

```

template <typename T>

std::optional<std::vector<T>> DeserializeVector(const std::string& data) const;

template <typename K, typename V>

std::optional<std::map<K, V>> DeserializeMap(const std::string& data) const;

template <typename K, typename V>

std::optional<std::unordered_map<K, V>> DeserializeUnorderedMap(const std::string& data) const;

```

Positioned Deserialization : This is the key method for custom type lambdas. It reads one value starting at pos and advances pos past the consumed bytes, so sequential calls parse fields from a concatenated string.

```

template <typename T>

std::optional<T> DeserializeAt(const std::string& data, size_t& pos) const;

```

Supported types via **if constexpr** dispatch **int, double, bool, char, std::string, std::vector<T>, std::map<K,V>, std::unordered_map<K,V>**. Unsupported types trigger a **static_assert** at compile time.

Custom Type API

// Register a type with serialize + deserialize lambdas

```

template <typename T>

void RegisterType(const std::string& type_id,

    std::function<std::string(const T&)> serialize_fn,

    std::function<std::optional<T>(const std::string&)> deserialize_fn);

```

// Check if a type is registered

```

bool IsTypeRegistered(const std::string& type_id) const;

// Serialize: wraps inner content → custom:<type_id>:<inner_len>:<inner>;

template <typename T>

std::string Serialize(const std::string& type_id, const T& value) const;

```

// Deserialize: strips envelope, passes inner content to registered lambda

```
template <typename T>
```

```
std::optional<T> Deserialize(const std::string& type_id, const std::string& data) const;
```

Custom Type Registration - Detailed Example

```
struct Agent {
```

```
    std::string name;
```

```
    double health;
```

```
    int level;
```

```
};
```

```
Serializer s;
```

```
s.RegisterType<Agent>("Agent",
```

// Serialize: concatenate primitive serializations in field order

```
 [&s](const Agent& a) -> std::string {
```

```
     return s.Serialize(a.name) + s.Serialize(a.health) + s.Serialize(a.level);
```

```
 },
```

// Deserialize: read fields positionally using DeserializeAt

```
 [&s](const std::string& data) -> std::optional<Agent> {
```

```
     size_t pos = 0;
```

```
     auto name  = s.DeserializeAt<std::string>(data, pos);
```

```
     auto health = s.DeserializeAt<double>(data, pos);
```

```
     auto level  = s.DeserializeAt<int>(data, pos);
```

```
     if (!name || !health || !level) return std::nullopt;
```

```
     return Agent{*name, *health, *level};
```

```
 }
```

```
);
```

// Usage

```
Agent agent{"Steve", 100.0, 7};  
std::string data = s.Serialize<Agent>("Agent", agent);  
// → "custom:Agent:20:s:5:Steve;d:100;i:7;;"  
auto restored = s.Deserialize<Agent>("Agent", data);  
// → Agent{name="Steve", health=100.0, level=7}
```

Error Handling

All deserialization returns `std::nullopt` on failure — no exceptions are thrown. `Serialize` returns "" for unregistered custom types.

Validation checks applied at each parse step: type tag prefix verification, input bounds checking, numeric parse error codes (`std::from_chars`/`strtod`), semicolon terminator validation, container count sanity check (`count > remaining/3 + 1`), and `std::bad_any_cast` handling for custom type mismatches.

Planned Improvements

Stream-based I/O — Add `ostream`/`istream` overloads so game state can be written directly to files instead of building one big string first.

Versioning and checksums — Prepend a version tag (e.g., `V1:`) and append a CRC32 checksum to protect against format changes and file corruption.

Broader type support — Add native support for commonly needed types like unsigned int, float, `std::pair`, and `std::set` so other teams don't have to register them as custom types.

Thread safety — The type registry isn't thread-safe. Document this limitation and consider adding a `std::shared_mutex` if multiple threads need to serialize concurrently.

Datum - Ian Wettlaufer

The `Datum` class is a flexible single value container that can hold a double, `std::string`, or boolean value, using an `std::variant` under the hood. This allows the user to manipulate data without needing to know the specific type.

Public Member Functions:

Constructors: A `Datum` object can be instantiated with either a double, bool, `std::string&`, `char*`, or with an empty value.

`bool IsDouble()`, `bool IsString()`, `bool IsBool()`:

Purpose: The Datum class implements type checker functions that allows the user to retrieve the original underlying type before becoming a Datum object by using the `std::holds_alternative<T>()` function.

Signatures:

`bool Datum::IsString() const`

`bool Datum::IsDouble() const`

`bool Datum::IsBool() const`

Return: Boolean value indicating if it is of the selected type

AsDouble()

Purpose: Can be called on any datum object and it will try to convert to a double

Signature: `double AsDouble() const;`

Return: Original value as a double or NaN()

Error conditions: If the underlying data is non-numeric or partially numeric the Datum class will return `std::numeric_limits<double>::quiet_NaN()` allowing for graceful failure.

AsString()

Purpose: Can be called on any datum object and it will convert to an `std::string`.

Signature: `std::string AsString() const`

Return: original value as an `std::string`.

Error conditions: None, all values should be able to convert to string.

AsBool()

Purpose: Convert original value to a bool, considering an empty string to be false, as well as the double value 0.0

Return: Original value as a boolean.

Operator overloads (+, -, *, /, ==, <, >, <=, >=)

Purpose: The datum class supports all basic arithmetic operations as well as comparison operations. These will automatically try to convert to the same type and perform comparison or math seamlessly.

Error conditions: If attempting to do math with non numeric strings, the Datum class will propagate NaN(). Division by zero will yield NaN, as well as any comparison involving NaN will propagate NaN allowing the program to fail gracefully. Mixed type comparisons will attempt numeric comparison first where possible, then lexicographical.

Private Helper Functions:

double Datum::NaN():

Purpose: Helper function that returns `std::numeric_limits<double>::quiet_NaN()`. Used for graceful error handling.

DataGrid - Mitchell McAuley

The DataGrid class contains a matrix of Datum objects and the functionality to quickly read, write, and dynamically expand the grid.

- Easy access - can request a single row, column, or element as needed.
- Data management - all basic functionality to managing a matrix of objects.
- Compatibility - transforms supported data types to Datum objects automatically to reduce unnecessary exposure with the Datum class.

Similar Standard Library Concepts

- `std::vector<std::vector<Datum>>`

Public Member Functions

- **Insert:**
 - **Purpose:** Can be called on any datum object and it will add it to the next available space in the grid. Also automatically expands the data structure when it runs out of space by adding a row.
 - **Signature:** `template <typename T> void Insert(T element);`
- **Insert**
 - **Purpose:** Can be called on any datum object and it will add it to the specified coordinates

- **Signature:** `template <typename T> void Insert(int r, int col, T element);`
- **Error conditions:** If the specified coordinate is outside the bounds of the grid, it throws an instance of `std::out_of_range`
- **Append**
 - **Purpose:** Add an entire row to the bottom of the grid
 - **Signature:** `void Append(const std::vector<Datum> & row)`
 - **Note:** Strictly takes a `std::vector` of `Datum` objects.
- **Row**
 - **Purpose:** Outputs a specified row for use.
 - **Signature:** `std::vector<Datum> Row(int r) const`
- **Column**
 - **Purpose:** Outputs a specified column for use.
 - **Signature:** `std::vector<Datum> Column(int c) const`
- **At**
 - **Purpose:** Returns the element at the given index.
 - **Signature:** `[[nodiscard]] Datum At(int r, int c) const;`
 - **Note:** There is no bounds check.
- **Find**
 - **Purpose:** searches the grid for a key and returns its indices
 - **Signature:** `[[nodiscard]] Datum At(int r, int c) const;`
 - **Note:** There is no bounds check.
- **begin**
 - **Purpose:** An iterator to the beginning of the grid
 - **Signature:** `Iterator begin()`
- **end**
 - **Purpose:** An iterator pointing to one space past the last element in the grid.
 - **Signature:** `Iterator end()`

Built in iterator

- Iterates through the data structure in row-major sequence, top left to bottom right.
- Only iterates to the last inserted element, not the full dimensions of the matrix.